ARKANOID (9-13) #For_beginners #Game_creation #Individual

Part 0. Before you begin

Required materials:
- Sample solution file to be demonstrated to students
- A4 sheets (1 sheet per student).

Teaching materials:
- ❏ Link to the test to pick up the appropriate projects.
- ❏ Link to the sample solution.

Recommended implementation time for the project: Two lessons.

Part 1. General project information

Minimal prior knowledge required from students		*Learning results*
Each student: - knows how to create sprites of any type in Play; - understands that additional files used in a project must be added to the project folder; - understands how to work with sounds in Pygame (knows the methods for background music and sound); - knows how to create lists with different data types; - understands when the methods clear() and append() should be used for the created lists; - has successfully implemented the "Piano" learning project and wants to develop it further.	**Arkanoid**	- Each student prepared a project checklist and worked according to the plan. - Each student applied his/her knowledge of lists in practice (used lists to store sprite blocks). - Each student created a user interface for "Arkanoid" and set up physics of the sprite ball and sprite racket. - Each student understands that in certain cases physics in a program can be too "heavy" (e.g. when creating blocks as physical objects). *Ideas*: - Each student created an arcade game on his/her own. - Each student got an interesting and visually appealing project, which would be cool to play after class.

Part 2. Description of the expected results

Program	*Features*
Arkanoid	● When the program is launched, the ball starts moving across the field. The racket is located at the bottom of the screen. The blocks to be destroyed are shown in the upper part. ● A block will be destroyed (and will disappear) once the ball touches it. ● The user can move the racket to the left or to the right by pressing "A" or "D". The ball jumps off the racket when touches it. ● If the ball misses the racket and falls on the "floor", the program will display "YOU LOSE". ● If the user destroys all the blocks without letting the ball touch the "floor", the program will display "YOU WIN". ● *Optional:* at the beginning of the game, the background music starts playing and plays until the end of the game. ● *Optionally*: Before starting the game, the user selects the difficulty level (it affects the ball speed). At the normal difficulty level, the ball moves slower than at the advanced one.

Part 3. Project stages

IMPORTANT!

Before picking up the projects, make sure to suggest the test for the students (link). To run the test, each student need to launch a browser and sign in to the platform, since AlgoVSCode does not support this test. The test includes basic theoretical questions about the topics studied in the project module and checks the corresponding practical skills of the students. Its results are for information only and should help the teacher understand whether the students are ready for harder tasks.

For students who made ***less than 3 mistakes***, we recommend trying out "Synthesizer" or "Arkanoid". For students who made ***3 or more mistakes***, we recommend getting back to the previous projects and picking up any remaining project of their choice.

However, the test cannot clearly figure out whether a particular student is ready for harder tasks or not. ***The teacher should make the final decision on whether to pick up "Synthesizer" or "Arkanoid" for a particular student only after a one-on-one interview with him/her.***

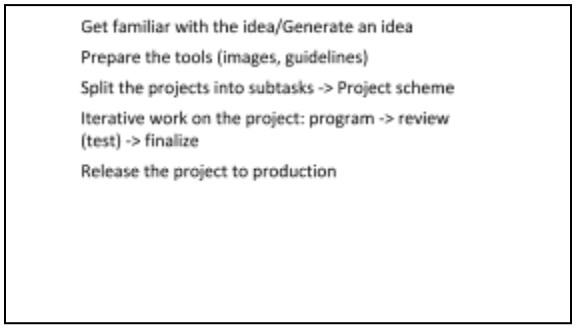
The goal of the project is to improve understanding of lists and finalize knowledge of the Play library. When developing "Arkanoid", students will use sprites as physical objects to create their first arcade game on Python (all their previous games were created in the learning projects only).

"Arkanoid" also features lists — a Python tool that the students have recently studied. The blocks[] list contains a set of sprite blocks that the player must destroy. Lists make it easy for a programmer to define the winning condition: if the blocks[] list is empty, than all the blocks are destroyed and the player has won.

Note. Drawing up a mind map in this project is optional and is recommended only for the students who are not good at such projects.

#	Stages and their goals	Content	Format	Results
1	**Readiness test "Play, lists, and for"** Pass the test on the platform. Count the mistakes, discuss them with the teacher and make the decision on whether to proceed to harder projects or not.	Start the lesson by introducing the pool of available projects. Remind students the "rules of the game": they have to pass a small test during this lesson to prove that they are ready for harder projects. Emphasize the fact that the test is not aimed at ranking the students by their knowledge. Tell them that it is very important to understand labor intensity of each "serious" project. If you pick up a task which is too labor-consuming, chances are that you will fail. Therefore you should always choose tasks of adequate difficulty and elaborate them without any hassle. And since we do not have lots of projects to choose from, let's try to figure out the average level using a test. Conduct the test for each student. Do not introduce	Individual work on the platform. Discussion of the results with the teacher.	Each student now understands his/her actual knowledge level thanks to the teacher and is ready to pick up a project.

		any time limits, but keep an eye on the students to make sure they do not help each other. Once you notice that a student has completed the test, ask him/her to come to your table (so that other students did not pay too much attention to your presence) and discuss the results. Make the decision on whether to proceed to harder projects or not. Remember that the teacher's opinion must prevail over all other factors. Pay special attention to students who made many mistakes. These are students for whom you need to decide whether they are ready for the next projects. For students who have successfully passed the test, suggest getting familiar with "Synthesizer: FS" and "Arkanoid: FS" and picking up one of them. **Note**. Even if the student has completely failed the test, make sure to emphasize that you will elaborate all the mistakes with him/her before getting started with the project. If we resolve all the issues now, we will prevent such mistakes in future.		
2	**"Projects fair"** Learn or repeat the project stages. Select a project from the pool of available projects. Get a functional specification (FS).	If you and the student decided to keep the current difficulty level, then proceed to the training manuals for "Screen pet" and "Gaming machine". Before picking up the projects, make sure to briefly remind the students of the project stages. Remember that revision will be effective if you only ask questions and prompt some of the answers, while the students are	Work with the teacher.	Each student has picked up a project. Each student has received and reviewed an FS, and is ready to get started.

	Review the requirements and get started with the project.	revising the stages by themselves.  Once the revision is over, everyone may choose a project for work. Then the students should sign in to the platform and review the functional specifications for the projects (**"Synthesizer: FS" or "Arkanoid: FS"**). Note that "Arkanoid" is easier than "Synthesizer", even despite the need for coding of the interaction among physical objects. "Synthesizer" requires deeper understanding of lists, while "Arkanoid" is designed mostly to revise the previously studied theoretical topics. After review, students should get started with the first task — create a project checklist.		

The description below relates only to "Arkanoid" project stages.

3	**Splitting a project into tasks.** Sign in to the platform. Find the	Arrange the creation of checklists by the students. Since they should already know how a typical checklist is created, feel free to allow them create it on their own.	Individual work, work with the teacher.	Students split their projects into tasks.

	second level of **"Arkanoid: FS"**(where the requirements to a checklist are shown). Assess whether familiar tools and theory are sufficient to complete the project (yes). Create a project checklist on an A4 sheet on your own or together with the teacher.	**The goal of each student is to describe his/her consecutive actions for the project implementation.** Each student must create a checklist on their own or in collaboration with another student who has also picked up "Arkanoid". Recommend students opening "Arkanoid: develop!" and read the first task "Create a project checklist". Answering the questions in this task will help students identify the key steps of their projects. Possible checklist of "Arkanoid" project: 1. Create a sprite player, sprite racket and multiple sprite blocks (the blocks must be created and added to the blocks[] list in the section @play.when_program_starts). 2. Implement the launch of the sprite player movements in @play.when_program_starts (add physics and define the initial speed). 3. Handle events in the section @play.repeat_forever: - Keyboard events: pressing "A" or "D" to move the racket to the right or to the left; - Touching of the player and the racket (the sprite player jumps off); - Touching of the player and the "floor" (the		The tasks are arranged and added to the checklist. A sample mind map for a project is shown in part 4 "Sample project implementation".

		game is lost -> display "YOU LOSE"); - Touching of the player and a block (the block is removed/erased). 4. Program the winning condition: if all the blocks are destroyed (i.e. the blocks[] list is empty), then display "YOU WIN". Remember that the more difficult it is for a student to program, the more detailed the work plan is needed. Remember that good students only need to list the stages of the program development (e.g. create sprites..., process events..., etc.), while others have to compose a detailed checklist with a description of each sprite and event.		

Note. If it is hard for a student to compose a checklist, then suggest making a mind map. A block diagram will not be suitable in this case, because "Arkanoid" has a complex structure. Try not to leave the student face to face with his/her problems. Whenever possible, sit down nearby and discuss the mind map together. Or try to find a pair for the student and organize their teamwork.

4	**Creating a sprite ball, sprite racket or sprite blocks.** **First launch of "Arkanoid".** Open the project draft: breakout.py.	Arrange individual work. Instruct a student to launch VSC, sign in to mars.algoritmika.org and then open "Arkanoid: VSC". The student must get started with the first point of the checklist — create a Play window and sprites. By that moment, the student will have all the knowledge and skills required for the creation of the player and racket. `# example of sprite-ball:`	Individual work, work in pairs * Creating blocks can be difficult. If the students are mostly 12-13 years old, then suggest them to collaborate and solve	"Arkanoid" with all necessary sprites, but without physics and event handling.

	Create sprites for UI elements.
	
	Launch the project, evaluate its visual appearance and eliminate artifacts.
		`ball = play.new_circle(`
		`    color='green', y = -160, radius = 15`
		`)`
		
		It is convenient to store blocks in a list. Once the ball touches a block, the block is erased. If there are no blocks left (the list is empty), then the game is over.
		
		`# list for blocks. Still empty.`
		`blocks[]`
		
		Blocks are rendered in the section @play.when_program_starts.
		
		Step 1. Suggest the student to start with a single block. It is necessary to create a new sprite called "block" and then add it to the blocks[] list.
		Step 2. The student needs to draw a row of blocks. Let's assume that drawing is done from left to right. At a certain point, e.g. at (block_x; block_y), it is necessary to carry out Step 1, and then increase the block_x coordinate by the sprite length while keeping the block_y coordinate unchanged. A new point is ready -> draw a new block.
		Step 3. Repeat Step 2 until the right border of the window is reached. The border can be retrieved using the method play.screen.right.
		Step 4. One row is ready. Let's draw 3 rows of blocks. Repeat Step 3 three times decreasing block_y by the block height in each iteration.
		
		Rendering of the blocks can be programmed as follows:
			the problem together.
			
			If the students are 9-10 years old, then become a moderator of the discussion. Organize them into a small group and direct their teamwork.
			
			Students may put their laptops side by side or use the same workstation.	

		<pre>#extreme borders of blocks
		    block_x = play.screen.left+75
		    block_y = play.screen.top-50
		
		    for i in range(3): #number of rows
		        while (block_x <= play.screen.right-30):
		        #block creation
		            block=play.new_box(
		                color='grey', x=block_x, y=block_y,
		width=110, height=30, border_color='dark grey',
		border_width=1
		            )
		            #adding the block to the list of blocks
		            blocks.append(block)
		            #shift by the block width and draw again
		            block_x=block_x + block.width
		
		        #define the start point for a new row
		        block_x=play.screen.left+75
		        block_y=block.y-block.height</pre>		

Though all the theoretical topics for the next stage are already known to students, using this knowledge in complex cases may require some assistance from the teacher.

5	**Define the ball and racket as physical objects.**	Arrange individual work. Now it is necessary to add physics to the section	Individual work, work in pairs.	When the program starts, all sprites are rendered and the

	Add physics to def start().
	
	Set the initial ball speed.
		@play.when_program_starts. The racket and ball in "Arkanoid" must be physical objects. The ball will be automatically jumping off the racket and flying at the specified speed.
		
		`#the platform is not affected by gravitation`
		`platform.start_physics(`
		`stable=True, obeys_gravity=False, bounciness=1, mass=1`
		`)`
		`#the ball randomly moves across the field at a speed of 30`
		`ball.start_physics(`
		`ball.start_physics(stable=False, x_speed=30, y_speed=30, obeys_gravity=False, bounciness=1, mass=10)`
		`)`
		
		It is not recommended defining blocks as physical objects, since they are too numerous. There is 99.9% probability that the program will not have enough time to process touching of the block by the ball and other events.
			*Consider arranging work in pairs when discussing the properties of physical objects. Sometimes students may forget the purpose of a given property. In this case, let the students experiment or remove gravitation, mass, etc. to identify the optimal configuration.	ball begins moving.

6	**Event handling in the game.**
	
	Handle the event of touching the block by the ball in the section @play.repeat_forever.
		The easiest way to get started is to begin with winning and losing situations. To do so, we need to create text sprites YOU LOSE and YOU WIN which will notify the user that the game is over.
		
		`#text sprite for lost games`
		`lose = play.new_text(words='YOU LOSE', font_size=100, color='red')`
			Individual work, work in pairs, work with the teacher.
			
			*You may help the student set up the physical speed of
				The program handles all necessary events. The project is ready for testing.
				
				The student

	Program the losing and winning conditions after the previous event. Display the corresponding message in each case.	Ask the student to stop here for a while and run the program. The student will see that text sprites are displayed at the very beginning and thus must be hidden until the end of the game. Do it in @play.when_program_starts.
		
		`lose.hide()`
		`win.hide()`
		
		Then it is necessary to write the corresponding conditional operators in the section @play.repeat_forever. If you have a group of advanced students, consider offering them individual tasks or teamwork at this stage. If the group is not good enough, then discuss with the students what conditions must be met.
		
		❏ Lost game: the ball touches the "floor" instead of the racket, i.e. the ball's Y coordinate is less than the corresponding Y coordinate of the racket.
		
		`#losing`
		`if ball.y <= platform.y:`
		`  lose.show()`
		
		❏ Won game: no more blocks are left, i.e. the length of the blocks[] list equals 0.
		
		`#winning`
		`if len(blocks) == 0:`
		`  win.show()`
			the ball and handle the event when the ball touches a block.	revised the material about the physical speeds of objects.

Then the student must define how the blocks will be destroyed once touched by the ball. Since blocks are not physical objects, the ball's behavior when it touches a block must be defined manually.

After touching a block, the ball jumps off in the opposite direction. To program this behavior, we only need to reverse the physical speeds (i.e. make them negative). Consider discussing it with the student, since physical speeds were studied about a month ago.

In general, the program must constantly iterate through all the blocks and check whether the ball has touched any of them. If a block has been touched by the ball, then the block must be hidden on the screen and removed from the list:

```
#removing the blocks
for b in blocks:
    if b.is_touching(ball):
        ball.physics.x_speed = -1 * ball.physics.x_speed
        ball.physics.y_speed = -1 * ball.physics.y_speed
        b.hide()
        blocks.remove(b)
```

And the final step is to program the platform management. This can be done in two ways:

Option 1. "Classic" method. This is how most students would implement it when working on their own. When the player presses "A" or another key, the platform is shifted by N blocks to the left.

```
#platform shift
if play.key_is_pressed('a'):
    platform.x -= 20
if play.key_is_pressed('d'):
```

		platform.x += 20
		
		Option 2. "Advanced" method. Consider suggesting this method to the most advanced students. Since the platform is a physical object as well, we can directly adjust its speed. When the "A" key is pressed, the platform moves to the left at a speed of N blocks. Note that the student must also handle the event in which no keys have been pressed.
		
		`#platform shift`
		`if play.key_is_pressed('a'):`
		`platform.physics.x_speed = -20`
		`elif play.key_is_pressed('d'):`
		`platform.physics.x_speed = 20`
		`else:`
		`platform.physics.x_speed = 0`		

Now the project can be delivered to the customer.

8	**Project delivery (testing by the teacher).**	Once a student says that the project is completed, arrange its testing. It is performed in three stages: - Testing by the developer. The student must open the project description and check if the required actions have been performed properly. - Peer review and testing by the students. To be performed similarly to code review. - Testing by the teacher. Check the program. If everything works fine, ask the student questions	Individual work, work in pairs, work with the teacher.	The basic part of the project is complete. If there is enough time left, the student may proceed to additional tasks.

		to understand his/her knowledge of the topic. If the student answers them successfully, offer him/her additional tasks.		

Part 4. Sample project implementation.

Project checklist:

1. Create a sprite player, sprite racket and multiple sprite blocks (the blocks must be created and added to the blocks[] list in the section @play.when_program_starts).
2. Implement the launch of the sprite player movements in @play.when_program_starts (add physics and define the initial speed).
3. Handle events in the section @play.repeat_forever:
    - Keyboard events: pressing "A" or "D" to move the racket to the right or to the left;
    - Touching of the player and the racket (the sprite player jumps off);
    - Touching of the player and the "floor" (the game is lost -> display "YOU LOSE");
    - Touching of the player and a block (the block is removed/erased).
4. Program the winning condition: if all the blocks are destroyed (i.e. the blocks[] list is empty), then display "YOU WIN".

Code

```
import play
from random import randint

#SCREEN
play.set_backdrop('light blue')
frames = 25 #frame rate
lose = play.new_text(words='YOU LOSE', font_size=100, color='red')
win = play.new_text(words='YOU WIN', font_size=100, color='yellow')
```

```
platform = play.new_box(color='brown', y = -250, width = 150, height = 15)
ball = play.new_circle(color='green', y = -160, radius = 15)
blocks = []


@play.when_program_starts
def start():
    #the platform is not affected by gravitation and is controlled only from the keyboard
    platform.start_physics(
        stable=True, obeys_gravity=False, bounciness=1, mass=1
    )
    #the ball is not affected by gravitation and randomly moves across the field
    ball.start_physics(
        ball.start_physics(stable=False, x_speed=30, y_speed=30, obeys_gravity=False, bounciness=1, mass=10)
    )

    #generating new blocks
    block_x = play.screen.left+75
    block_y = play.screen.top-50

    for i in range(3): #number of rows
        while (block_x <= play.screen.right-30):
        #for i in range(5):
            block=play.new_box(
                color='grey', x=block_x, y=block_y, width=110, height=30, border_color='dark grey', border_width=1
            )
            blocks.append(block)
            block_x=block_x + block.width
```

```
        block_x=play.screen.left+75
        block_y=block.y-block.height

    platform.show()
    lose.hide()
    win.hide()

@play.repeat_forever
async def game():
   #platform shift
    if play.key_is_pressed('a'):
        platform.physics.x_speed = -20
    elif play.key_is_pressed('d'):
        platform.physics.x_speed = 20
    else:
        platform.physics.x_speed = 0

    #removing the blocks
    for b in blocks:
        if b.is_touching(ball):
            ball.physics.x_speed = -1 * ball.physics.x_speed
            ball.physics.y_speed = -1 * ball.physics.y_speed
            b.hide()
            blocks.remove(b)

    #losing
    if ball.y <= platform.y:
        lose.show()
```

```
    #winning
    if len(blocks) == 0:
        win.show()

    await play.timer(seconds=1/frames) #frame rate (a pause after each several frames)

play.start_program()
```

Part 5. Methods for evaluating the results

Stage 1. *Program operability check*

Any program must handle the following test cases. If all the tests are passed, this means that the student has achieved the mandatory basic level of knowledge.

Program testing	
• The program is launched, no keys have been pressed yet.	• The ball moves freely across the field and destroys blocks by touching them. Once the ball touches the "floor", the following message is displayed: "YOU LOSE". If the ball has destroyed all the blocks, the user will see the message "YOU WIN".
• The user starts pressing the "A" or "D" keys.	• The platform starts moving to the right or to the left respectively. At the same time, the ball continues to move freely.

Stage 2. *Checking the student's knowledge level.*

Note. The teacher should use this section for assessing the overall knowledge of the group instead of public personal assessment of each student.

The "Arkanoid" project does not require any sophisticated methods. Instead, this program summarizes the knowledge obtained by students in the project module. This is why a detailed knowledge check is not required at this stage. However, you may ask one of the following questions.

Checking the student's knowledge level	
• *Can the ball destroy several blocks at once? Why?*	• Yes. The handler for block touching events does not require that the ball touches a block only from below. We have defined a general case when sprites touch each other in any way.
• *There are 2 ways to move the racket:* 1. `if play.key_is_pressed('a'): platform.x -= 20` 2. `if play.key_is_pressed('a'): platform.physics.x_speed=-20` *What is the difference? Will the program look different?*	• In the first case, we simply move the sprite by coordinates. In the second case, we set the physical speed for the sprite. • In the first case, pressing the "A" key once will move the sprite by 20 pixels to the left and then stop it. In the second case, the sprite will be assigned a constant speed at which it will continue moving even after the user releases the key.

Part 6. Additional tasks for project improvement

Consider offering additional tasks to the students who want to improve their projects.

Task 1. Your game will become even more interesting when you add a counter of lives. Create a text sprite and add it to the place where it won't not interfere with the gameplay. Initialize the counter (e.g. assign it the value 3). The counter's value must be decreased by 1 each time the ball touches the "floor". If the counter's value becomes zero, the player will see the message "YOU LOSE" and the game will end.

Task 2. Set the background music! Download a music file from the Internet and add it to the project using the "Add file" tab. Then load Pygame, add the music file to the program and enable the music in the section @play.when_program_start.

ARKANOID #For_beginners #Games_development #Individual

Dear developer, you have been assigned a task to develop the **ARKANOID** program.

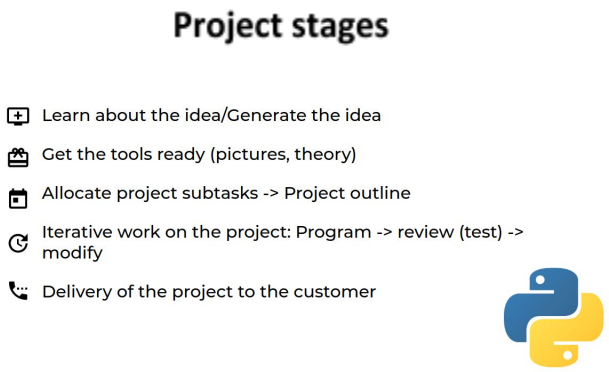
"Arkanoid" must perform the following:

- At the startup, the program must create a ball, a racket and a set of blocks to be destroyed by the player.
- It is necessary to define how the ball moves across the field, jumps off the racket and touches a block (when the ball touches a block, the block is destroyed).

The player will lose the game if the ball touches the "floor" at least once before all the blocks are destroyed (the message "YOU LOSE" will appear).
The player will win if the ball destroys all the blocks without touching the "floor" (the message "YOU WIN" will appear).

It is recommended to implement this program in Python **over 2 lessons.**
Testing is a critical stage of software development. It consists of 3 steps:

- *Check the program operability on your own*. E.g. make sure to check the following:
  - Will a block be destroyed if the ball touches it?
  - Will the message "YOU LOSE" appear if the ball touches the "floor" (i.e. misses the racket and fails to jump off it)?
  - Will the message "YOU WIN" appear if all the blocks are destroyed?
- *A developer peer review* performed as follows:



After successful testing, the project can be presented to the customer. This is the final stage of the project testing.